Data Structures Programming Assignment

Doubly Linked Lists • Circular Linked Lists • Stacks • Queues

Scenario Theme: Applied Data-Structure Systems in C++

Course:	Data Structures	Course Code: CS 2001

Program:	**BS(CS)**	**Semester:**	**FALL-2026**
Due Date:		**Total Marks:**	**100**
Type:	**Assignment 2**	**Page(s):**	**11**
Course Instructor	**Rana Waqas Ali**	**TA.**	**Syed Aoun Haider Sherazi**

Important Instructions:

1. Submit your .cpp files named as your roll number+QuestionNo+FileNo. Dont submit zip files.
2. You are not allowed to copy solutions from other students. We will check your code for plagiarism using plagiarism checkers. If any sort of cheating is found, negative marks will be given to all students involved.
3. Late submission of your solution is not allowed.
4. All questions / parts must be done keeping in mind the time constraint of that Data Structures or marks will be deducted.
5. In case of any confusion, feel free to discuss with Instructor or TA.
6. Do proper edge cases handling for all questions.

Read the Mini-C++ specification for Question 1 and the implementation restrictions before starting. Question 2 has its own input-file rules.

1. Learning Outcomes

- Implement stacks, queues, doubly linked lists, and circular linked lists using dynamic nodes and pointers.
- Select a data structure because of the operation it supports, not merely because the assignment asks for it.
- Parse a restricted source file, validate structure, and evaluate integer expressions without using a real C++ compiler as the solution engine.
- Handle boundary cases such as empty structures, single-node circular lists, mismatched delimiters, invalid variables, and removal during traversal.
- Design modular C++ classes and produce deterministic diagnostic output suitable for testing.

2. Mini-C++ Language Used in Question 1

Question 1 uses a deliberately restricted subset of C++. You are not being asked to implement the full C++ grammar. Only the following statement forms must be recognized:

Statement Type	Accepted Form	Example
Declaration	int <identifier>;	int total;
Declaration + initialization	int <identifier> = <expression>;	int total = 4 + 6;
Assignment	<identifier> = <expression>;	total = total * 2;
Output	cout << <expression>;	cout << total + 1;
Block delimiter	{ or }	{
Single-line comment	// anything after // is ignored	int x = 5; // initial value

2.1 Expression Rules

- Values are signed integers only. No float, double, char, string, arrays, pointers, functions, conditionals, or loops are required.
- Expressions may contain integer constants, previously declared variables, parentheses, and the operators +, -, *, /, and %.
- Use normal precedence: parentheses first, then *, /, %, then +, -. Operators at the same precedence are left-associative.
- You may assume test data will not intentionally overflow a 32-bit signed integer.
- Division and modulo use integer semantics. Division or modulo by zero must be reported as an execution error.
- A semicolon is mandatory for declaration, assignment, and cout statements.

2.2 Common Error Conditions

Category	Examples
Structural	Mismatched (), [], or {}; closing delimiter with no matching opener; file ends while an opener is still unmatched.
Statement syntax	Missing semicolon, unsupported statement, malformed identifier, or malformed expression.
Semantic	Duplicate declaration, use before declaration, use of an uninitialized variable.
Runtime	Division by zero or modulo by zero.

3. Mandatory Implementation Restrictions

- Implement your own Stack, Queue, Doubly Linked List, and Circular Linked List classes using nodes/pointers.
- Do NOT use std::stack, std::queue, std::deque, std::list, std::forward_list, or another STL container as a replacement for any required core structure.

- std::string, fstream/ifstream, sstream, iostream, and basic helper functions are allowed. A small temporary string/token helper is acceptable, but the assessed data structures must remain your own implementations.
- Do NOT invoke g++, clang++, system(), an online compiler, or another external compiler/interpreter to decide whether the input file is valid.
- Do NOT hard-code the supplied examples. Your program must work for unseen valid and invalid input files following this specification.
- Memory allocated with new must be released correctly. Destructors and safe deletion logic are part of code quality.

Recommended Class Separation
Typical classes may include Stack<T>, Queue<T>, SymbolTableDLL, Statement, ExpressionEngine, Analyzer, PlayerNode, CircularPlayerRing, Challenge, ChallengeQueue, and MatchHistoryDLL. You may choose different names if responsibilities remain clear.

Question 1 — CodeSentry: Mini-C++ File Analyzer & Integer Interpreter [50 marks]

Scenario. A lightweight teaching IDE called CodeSentry receives a student's .cpp file. Before showing any result, it must check whether the source is structurally valid under the Mini-C++ rules, then execute supported integer statements in order. Your task is to implement the analysis pipeline without using a real C++ compiler.

Required System Behaviour

1. Read the source file from disk. Preserve the original line number of every non-empty logical line.
2. Remove the // comment portion before analysis. A line that becomes empty may be ignored.
3. Perform a delimiter scan using your custom Stack. Validate (), [], and {}. When an error occurs, report the line number, column number, actual delimiter, and expected delimiter where applicable.
4. Enqueue executable statements into your custom Queue so that they are processed strictly in source order. Standalone { and } lines are structural only and do not need to enter the execution queue.
5. Validate each queued statement against the permitted Mini-C++ forms. Reject malformed statements or missing semicolons with the original source line number.
6. Maintain the variable/symbol table as a Doubly Linked List. Each symbol node must store at least: variable name, initialized flag, current integer value, and declaration line. Variables must be printed once forward and once backward at the end to demonstrate correct prev/next links.
7. Evaluate every arithmetic expression using stacks. A recommended approach is infix-to-postfix conversion with an operator stack followed by postfix evaluation with a value stack. Direct recursive evaluation is not accepted as a substitute for the stack requirement.
8. For cout << expression;, evaluate the expression and append the resulting integer to the program output section of the report.
9. Stop execution at the first statement-level syntax, semantic, or runtime error. Structural delimiter errors may be detected before statement execution begins.

Required Data-Structure Roles

Structure	Required Role	Minimum Evidence in Output/Code
Stack	Delimiter matching; expression conversion/evaluation	Push/pop/peek logic implemented by you; mismatch location reported.
Queue	Stores statements awaiting execution	Statements execute in FIFO source order.
Doubly Linked List	Variable/symbol table	Forward and reverse traversal of final symbol table.

Illustrative Input File: q1_sample.cpp

```
{
    int b = 5;
    int c = 6;
    int a = b + c;
    a = (a * 2) - 3;
    cout << a;
}
```

Your report for a valid file should clearly show: structural status, statement execution trace, cout output, and the final symbol table in both traversal directions.

Minimum Diagnostic Examples Your Program Must Handle

- int a = (5 + 3]; → delimiter mismatch.
- int x = 5 → missing semicolon.
- int x = y + 1; → y used before declaration.
- int x; cout << x; → x is declared but uninitialized.
- int x = 4 / (2 - 2); → division by zero.
- int x = 1; int x = 2; → duplicate declaration when written as separate statements/lines.

Required Output Format (content, not exact spacing)

```
FILE: q1_sample.cpp
STRUCTURAL CHECK: PASS

EXECUTION TRACE
Line 2: int b = 5;              -> OK
Line 3: int c = 6;              -> OK
Line 4: int a = b + c;          -> OK
...

PROGRAM OUTPUT
<integer result(s)>

SYMBOL TABLE — FORWARD
<name, value/uninitialized, declaration line>

SYMBOL TABLE — BACKWARD
<same nodes traversed using prev links>

FINAL STATUS: SUCCESS
```

Question 1 Deliverables

- Source code for the analyzer/interpreter.
- At least 8 test files: minimum 3 valid and 5 invalid, covering different categories of failure.
- A short README explaining compilation, execution command, class responsibilities, and any assumptions within the stated Mini-C++ rules.

Question 1 Marking Breakdown

Component	Marks
Custom stack + delimiter diagnostics	8
FIFO statement queue and line preservation	5
Mini-C++ statement parsing and syntax checks	7
Stack-based expression conversion/evaluation	10

Doubly linked symbol table + forward/reverse traversal	7
Semantic/runtime error handling	6
Output clarity, modular design, memory management, testing	7

Question 2 — CodeArena: Circular Coding Challenge Tournament [50 marks]

Scenario. The TGS programming lab is running a CodeArena practice tournament. Several teams are seated in a fixed circular turn order. A central deck of arithmetic challenge cards is processed in first-in, first-out order. On every turn, the current active team receives the next challenge. Your program must compute the correct integer result using stacks, compare it with the submitted answer stored on the card, update the team's score/strikes, record the event, and move to the next active team. A team is removed from the circle after its second incorrect valid answer. The system must continue correctly even when the head, tail, middle, or only remaining circular node is removed.

Input Files

The simulator reads two plain-text files. Blank lines may be ignored.

players.txt — one player/team per line in the form <playerID>|<displayName>. Player IDs must be unique.

```
P01|Alpha
P02|ByteForce
P03|CodeCrew
```

challenges.txt — one challenge card per line in the form <challengeID>|<integer expression>|<submitted answer>. The submitted answer is the recorded answer for whichever active team receives that card.

```
C01|(8+4)*2|24
C02|20/(2+3)|5
C03|7+3*4|19
```

Question 2 expression grammar. Expressions contain integer constants, parentheses, and the operators +, -, *, /, and %. Variables are not used in Question 2. Use normal arithmetic precedence and left associativity. Division and modulo use integer semantics. You may assume numeric literals themselves are non-negative; intermediate or final results may be negative.

Tournament Rules

- Every team starts with score = 0 and strikes = 0.
- A correct answer adds 10 points and does not add a strike.
- An incorrect answer to a valid challenge adds one strike. No points are awarded for that card.
- When a team reaches 2 strikes, it is ELIMINATED immediately after that turn and its node must be removed safely from the Circular Linked List.
- If a challenge card itself is invalid (for example mismatched parentheses, malformed expression, division by zero, or modulo by zero), mark the card INVALID_CHALLENGE. It must not change the current team's score or strikes.
- After every processed card, advance to the next active team. If the current team was eliminated, continue from the eliminated node's former successor.
- The tournament ends when the challenge queue becomes empty or when no active teams remain. If one team remains, its CLL node must continue pointing to itself until the tournament ends or that team is eliminated.

Phase A — Build the Queue and Circular Player Ring

1. Read players.txt and insert the teams into your custom Circular Linked List in file order. The last player node must link back to the head.

2. Each player node must store at least: player ID, display name, score, strikes, and next pointer.

3. Read challenges.txt and enqueue all challenge cards into your custom Queue in exactly the same order as they appear in the file.

4. Each challenge node must store at least: challenge ID, expression text, submitted integer answer, and next pointer.

5. Reject malformed player/challenge input lines with a clear message. Duplicate player IDs must also be detected.

Phase B — Stack-Based Challenge Engine

6. When a team receives a challenge, first validate the expression's parentheses using your custom Stack. Report the challenge ID and the location/type of any mismatch.

7. For a structurally valid expression, use stacks to evaluate it. A recommended design is infix-to-postfix using an operator stack, followed by postfix evaluation using a value stack.

8. Directly calling another expression evaluator, scripting engine, or compiler is not allowed. The arithmetic result must come from your own stack-based logic.

9. Detect malformed operator/operand sequences as well as division or modulo by zero. Such a card becomes INVALID_CHALLENGE and does not penalize the player.

Phase C — Circular Turn Management and Safe Elimination

10. Set current to the head player. Dequeue exactly one challenge for the current player's turn.

11. If the card is valid, compare the computed result with its submitted answer and update score/strikes according to the tournament rules.

12. If the player is not eliminated, advance current = current->next.

13. If the player is eliminated, remove the current node without breaking the circle. Update head/tail/current correctly when deleting the first node, a middle node, the last node, or the only remaining node.

14. After each turn, print or record the active circular order starting from head. This is required evidence that the ring remains valid after insertions and deletions.

15. Do not rebuild the entire circular list after an elimination. Perform the pointer updates on the existing nodes.

Phase D — Match History (Doubly Linked List)

Maintain a separate Doubly Linked List containing one history node for every processed challenge card. **Each history node must store at least:**

- turn number,
- player ID and display name,
- challenge ID and expression,
- computed result or INVALID,
- submitted answer,
- outcome (CORRECT / WRONG / INVALID_CHALLENGE / ELIMINATED), and
- the player's score and strike count immediately after the turn.

When the tournament ends, traverse this history once from head to tail and once from tail to head. The reverse report must follow prev pointers; copying the data into another structure and reversing it is not accepted.

Required Data-Structure Roles

Structure	Required Role	Why It Fits the Scenario
Circular Linked List	Active player/team turn ring	Turns repeat naturally; players may be removed while traversal continues.
Queue	FIFO challenge deck	The earliest challenge loaded must be the next challenge attempted.
Stack	Parenthesis validation and arithmetic evaluation	LIFO behavior supports nested parentheses and operator precedence.
Doubly Linked List	Complete match history	Supports chronological and reverse chronological traversal using next/prev links.

Illustrative Example

For the following input, teams receive challenge cards one at a time in circular order. The values below are illustrative test data; your program must work for unseen files.

```
players.txt                    challenges.txt
P01|Alpha                      C01|(8+4)*2|24
P02|ByteForce                  C02|20/(2+3)|5
P03|CodeCrew                   C03|7+3*4|19
                               C04|(10-2)%3|2
                               C05|6*(5-1)|23
                               C06|18/(3*2)|3
                               C07|9+1|8
                               C08|(2+3]*(4)|20
                               C09|30/(5-5)|0
                               C10|11-6|5
```

The beginning of the run should therefore include: Alpha answers C01 correctly; ByteForce answers C02 incorrectly because the computed result is 4; CodeCrew answers C03 correctly; and later ByteForce is eliminated if it receives a second valid incorrect card. C08 is structurally invalid and C09 causes division by zero, so those cards must not add strikes to whichever teams receive them.

Required Final Report

```
LOAD REPORT
Players loaded: <n>
Challenges queued: <m>

TURN TRACE
Turn 1 | P01 Alpha | C01 | result=24 | submitted=24 | CORRECT | score=10 strikes=0
Active ring: P01 -> P02 -> P03 -> (back to P01)
Turn 2 | ...
...

FINAL ACTIVE RING
<remaining players in circular order, or EMPTY>

MATCH HISTORY — FORWARD
<history nodes from head to tail>

MATCH HISTORY — BACKWARD
<same nodes traversed from tail using prev links>

FINAL STATUS: CHALLENGES_EXHAUSTED / NO_ACTIVE_PLAYERS
```

Mandatory Edge Cases for Testing

- players.txt is empty.
- Exactly one player is loaded; its next pointer must point to itself.
- challenges.txt is empty.
- The head/current player is eliminated and head must move safely.
- A middle player and the tail player are eliminated in separate tests.
- The only remaining player is eliminated, leaving an empty circular list.
- A challenge contains mismatched or extra parentheses.
- A challenge causes division by zero or modulo by zero.
- The challenge queue becomes empty while multiple players are still active.
- All players are eliminated before all queued challenges are consumed.
- A malformed input record or duplicate player ID is encountered.

Question 2 Deliverables

- Source code for the complete CodeArena simulator.
- players.txt and challenges.txt plus at least 5 additional test sets covering normal play, invalid challenge cards, and different CLL elimination cases.
- A README section explaining your PlayerNode, ChallengeNode, history DLL node, circular insertion/removal logic, and stack-based expression evaluation approach.
- A short complexity note stating the time complexity of stack push/pop, queue enqueue/dequeue, DLL append/traversal, CLL insertion, and CLL removal when the necessary current/previous pointers are already known.

Question 2 Marking Breakdown

Component	Marks
Custom stack: validation + expression conversion/evaluation	10
FIFO challenge queue and input preservation	6
Circular player ring construction and correct traversal	9

Component	Marks
Safe CLL elimination: head/middle/tail/one-node cases	8
Doubly linked match history + forward/reverse traversal	7
Tournament rules, scoring/strikes, and invalid-card handling	4
Diagnostics and mandatory edge-case handling	3
Code quality, testing, memory management, complexity note	3

4. Submission Requirements

- Submit one ZIP file named: RollNo_DS_Assignment.zip.
- The ZIP should contain separate folders Q1 and Q2, source/header files, all test input files, and README.txt or README.md.
- Your code must compile with a standard C++17 compiler.
- Do not submit executable/binary files unless specifically requested by the instructor.
- Programs that do not read external input files, or that only work for the examples printed in this handout, will lose substantial marks.

5. Code Quality Expectations

- Use meaningful class, function, and variable names.
- Keep parsing, data-structure operations, turn-management logic, expression evaluation, and reporting reasonably modular.
- Check empty/full logical conditions before pop/dequeue/delete operations.
- Avoid memory leaks, double deletion, and dangling current/head/tail pointers.
- Comments should explain non-obvious logic rather than narrate every line of code.
- Error messages should be specific enough for a student user to understand what failed and where.

6. Overall Mark Summary

Question	Marks
Question 1 — CodeSentry	50
Question 2 — CodeArena	50
TOTAL	**100**

Assessment Note

This assignment evaluates data-structure implementation and integration. A solution that replaces the required structures with STL containers may produce correct output but will not satisfy the core learning objective.

— End of Assignment —